作1.12　假定利用增加向量模块来提高计算机的运算速度。计算机处理向量的速度比其通常的运算要快20倍，将可用向量处理部分所花费的时间占总时间的百分比称为可向量化百分比。

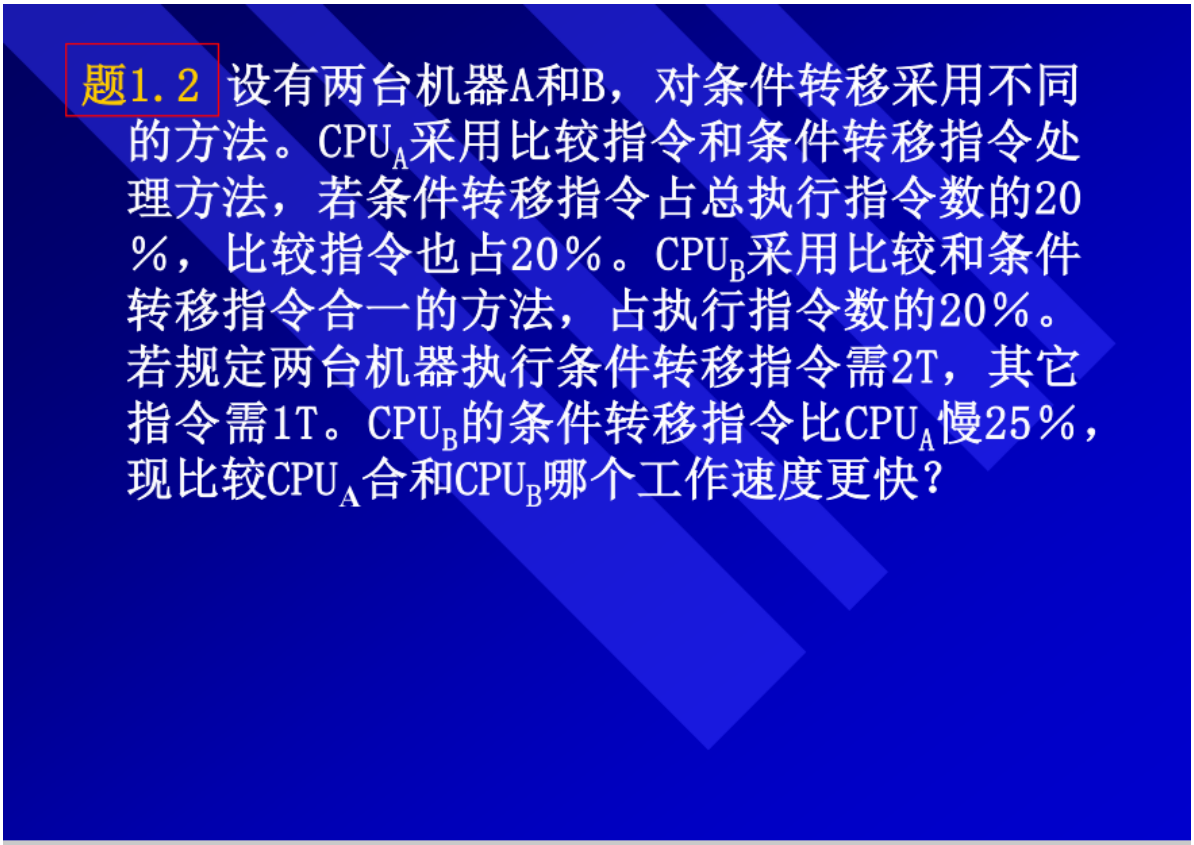
（1）求出加速比S和向量化百分比之间的关系式

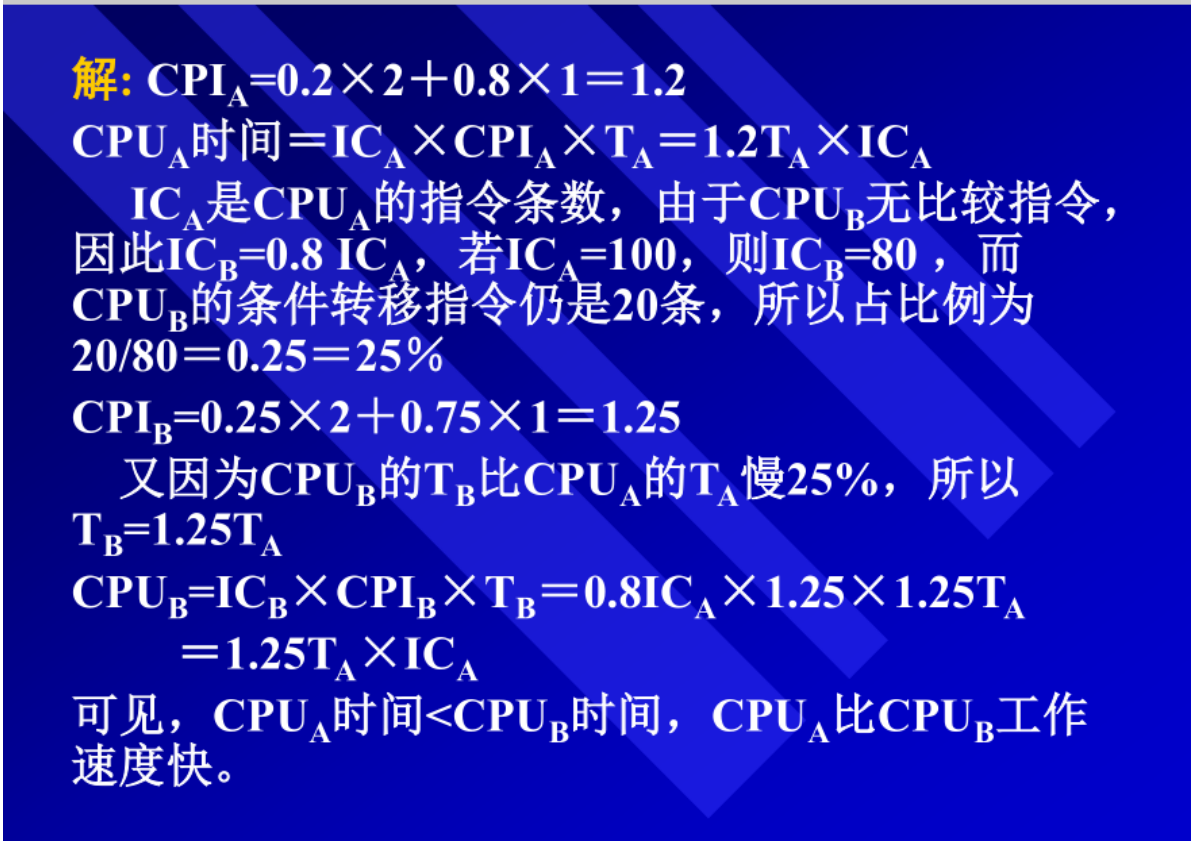
作1.13　（2）当要得到加速比为2时的可向量化百分比F为多少？

作1.14　（3）为了获得在向量模式所得到的最大加速比的一半，可向量化百分比F为多少？

加速比=加速部分占比+加速倍数

加速部分占比=向量化百分比=cache命中率=各种高性能部件占比





CPU执行时间=指令数×系统CPI×时钟周期

CPI的计算无需关注具体的频率或者时钟周期是多少, 只需要关注执行一条指令需要多少个周期即可. 例如题中执行一个转移指令需要2个时钟周期, 则计算CPI的时候用2就行了, 不需要考虑具体的T是多少.

指令数:

- CPUA: 100
- CPUB: 80

CPI:

- CPUA: 0.2*2+0.8*1
- CPUB: (2/8)*2 + 6/8 * 1

时钟周期T:

- CPUA : T
- CPUB: 1.25T

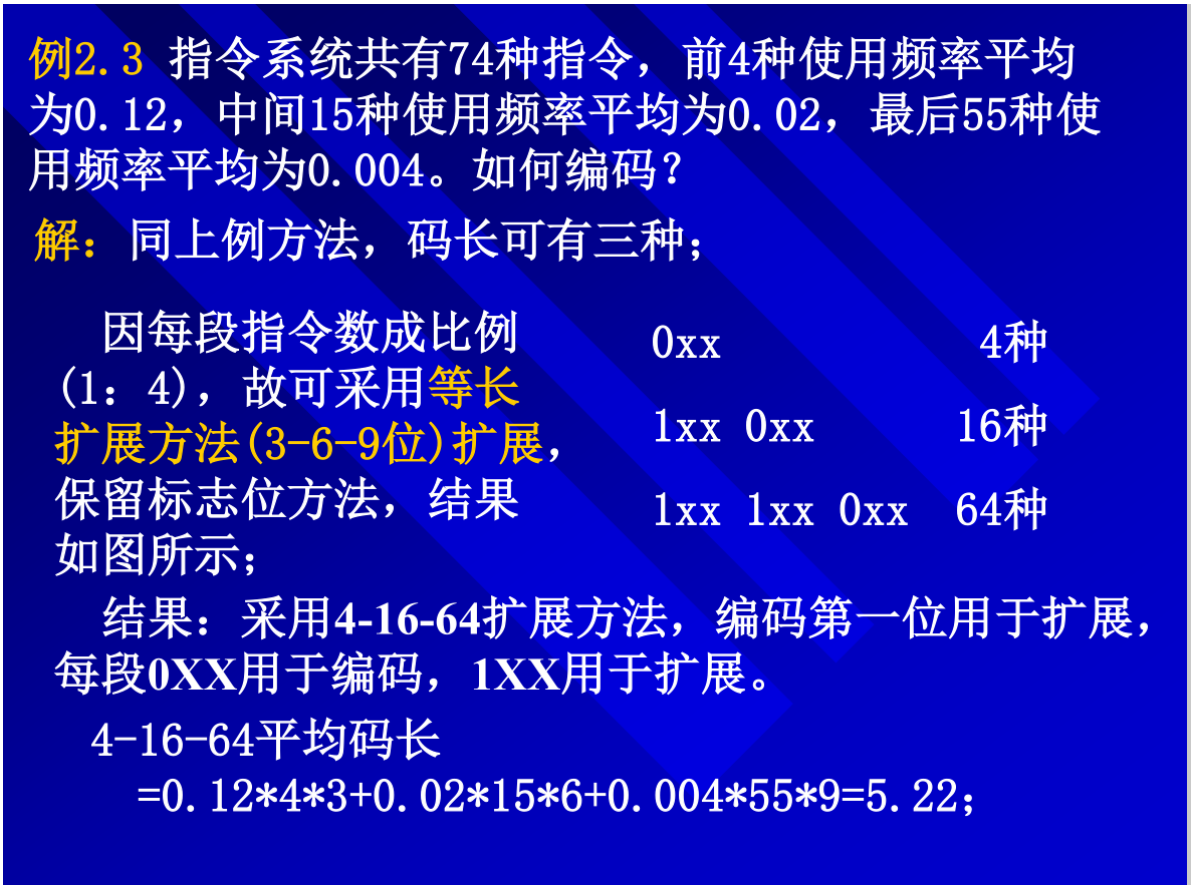
这个题可以体会到CISC和RISC的性能差异:

- CISI的指令个数小于RISC
- RISC的CPI小于CISC
- RISC的时钟周期T小于CISC
- 最终的执行总时间需要看**指令数量 vs CPI和T**

CPI与系统结构、指令集、计算机组织有关，而与时钟频率无关。

这是因为一条指令所需要的操作数是一定的，那么需要的时钟数就是确定的，与完成指令的一个执行步骤所用时间（即时钟频率）无关

8/64/512的变种: 4/16/64



给出数量和频率, 问如何编码?

考点: 结合频率的编码, 考虑扩展编码.

1. 看有几种频率, 计算一些数量比例: 4:15:55, 看看近似是否成比例. 4:16:64.
2. 如果为等比数列, 则可用定长扩展编码(扩展标志位), 扩展增幅为比例增幅: n = log(d)
3. 如果不等比, 则考虑变长扩展编码(前缀编码), 先看频率最大的指令有多少条, 进而确定第一种指令的位数. 然后没用完的编码作为第二种指令的前缀编码.

例2.4　指令系统共有78种指令，前10种使用频率平均为0.049，中间18种使用频率平均为0.02，最后50种使用频率平均为0.003。如何编码？

解：同上例方法，码长可有三种；因每段指令数比例为1：2：5，故不可采用等长扩展，采用不等长编码（ 4-6-10位）较能减少平均码长。

第一种采用4位编码中前10种(0000~1001)；

第二种采用第一种频率编码中的后5种编码(1010~1110)与扩展的2位共20种编码；

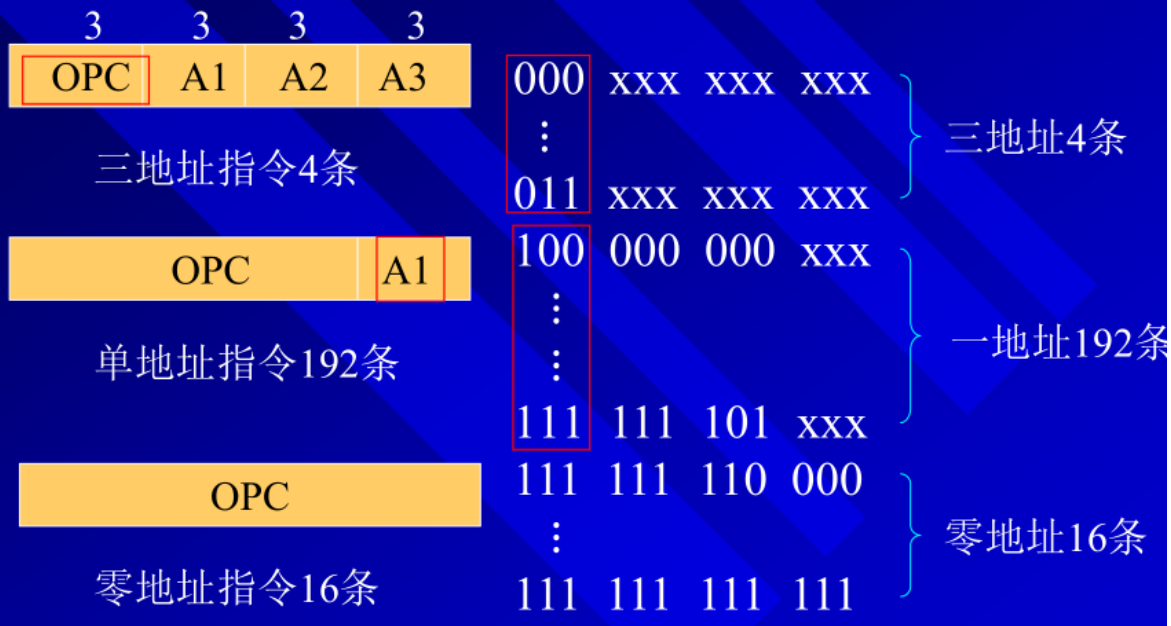
第三种采用第一种频率编码中的最后一种(1111)与扩展的6位共64种编码。



根据指令长度和地址码长度反推出操作数位数

例2.5 若某机要求有：三地址指令4条，单地址指令192条，零地址指令16条。设指令字长为12位，每个地址码长3位。问能否以扩展操作码为其编码？

解： 1. 三种指令格式字如下：



理解变址寻址

(2)

设计 8 位字长的寄存器-寄存器型指令如下：

因为只有 8 个通用寄存器，所以寄存器地址需 3 位，操作码只有两位，设计格式如下：

2	3	3
操作码 OP	源寄存器 R1	目的寄存器 R2

三条指令的操作码分别为 00、01、10。

设计 16 位字长的寄存器-存储器型变址寻址方式指令如下：

4	3	1	8
操作码 OP	通用寄存器	变址寄存器	偏移地址

四条指令的操作码分别为 1100、1101、1110、1111。

例 3.3

1. 计算内存容量=64x8bit=2^9

假若一计算机的主存储器按64块组织，块大小为8个字，高速缓存有8块。试按下述要求画图回答问题。

（1）画出全相联映像示意图、指定标记字段和主存地址格式。

（2）画出直接映像示意图、指定标记字段和主存地址格式。

（3）画出2路组相联映像示意图、指定标记字段和主存地址格式。

115

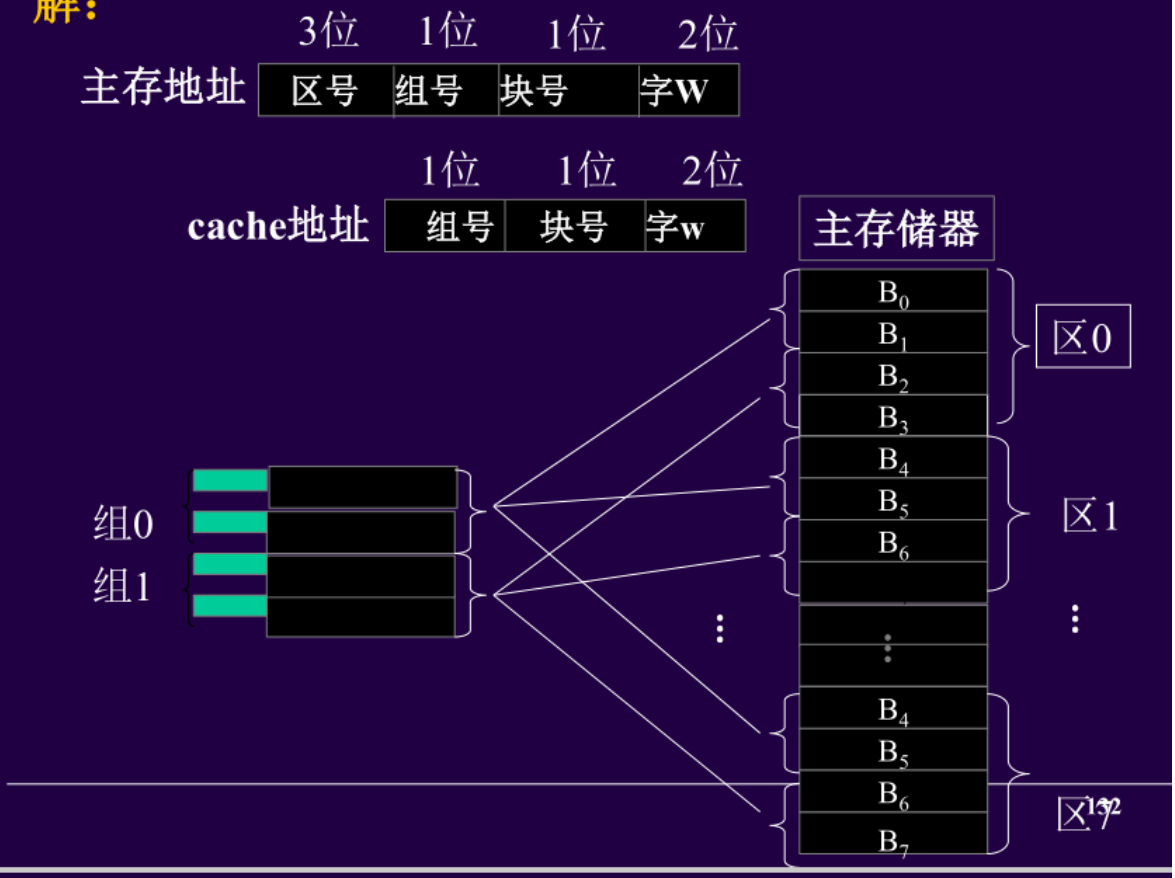
主存地址=主存块地址+块内偏移

主存块地址=

- 组相连: 区号+组号+组内块号 (区号+组内块号=tag; 组号=index)
- 直接相连: 区号+区内块号 (区号=tag; 区内块号=index)
- 全相连: 块地址

通过index去查询表格, 如果表格项与tag匹配且标志位为1, 说明cache中现在有这个块.

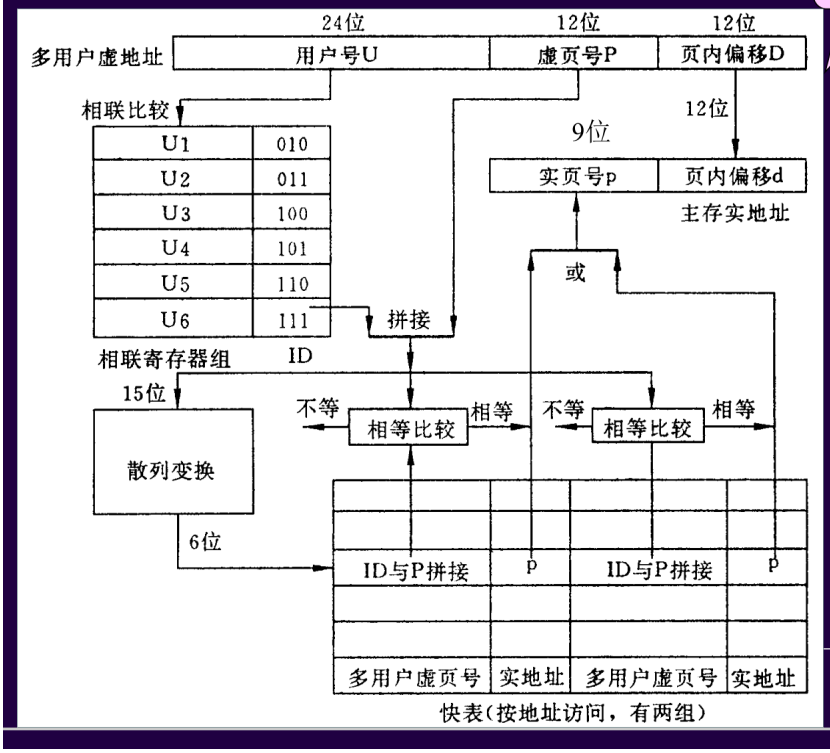
解：



（2路组相联FIFO）

时间 t	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
字地址流	1	14	50	89	20	17	19	56	19	11	14	43	15	16	9	17
组0	1	1	1	1*	20	20*	19	19	19	19	19	19	19	19	19*	17
			50	50	50*	17	17	17	17	17	17	17	17*	16	16	16
组1		14	14	14	14	14	14*	56	56	56*	14	14*	15	15	15	15
				89	89	89	89	89	89*	11	11*	43	43	43*	9	9
命中1次	调进	调进	调进	调进	替换	替换	替换	替换	命中	替换	替换	替换	替换	替换	替换	替换

例3.7 虚拟存储器举例—IBM37



2^{12}=4K

2^{12}=4KB

用户

每 页数？

每 30位

15位

64行

24+24位

相等

相等

快表行数？

快表每行的位数？

如何减少散列冲突？

快表为何采用两组？

相等比较的作用？

例 3.8

有1K个任务，短期内只有4个用户，每个用户程序空间可达4096页，每页512B，主存地址长度为20位，虚实地址经快表变换，问相关设计：

（1）实页位数 ？

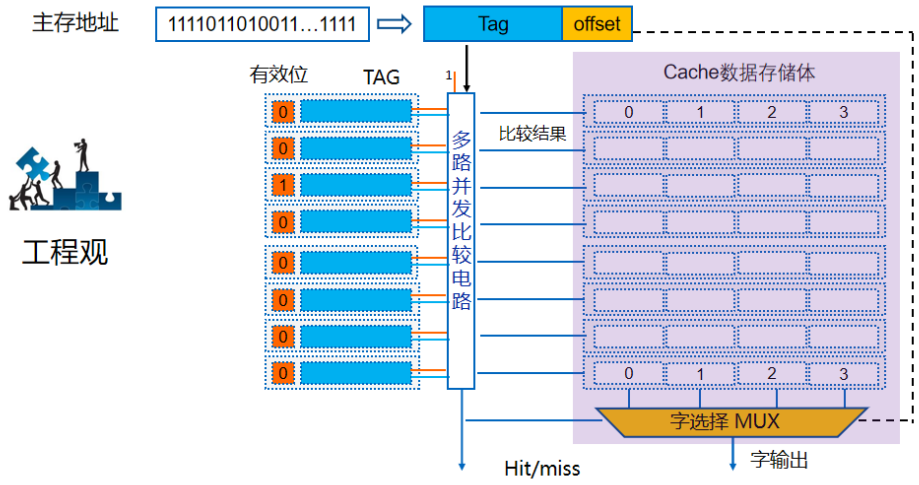
（2）每个相联寄存器的相联比较位数

（3）每个相联寄存器的总位数

（4）散列变换硬件的输入和输出位数

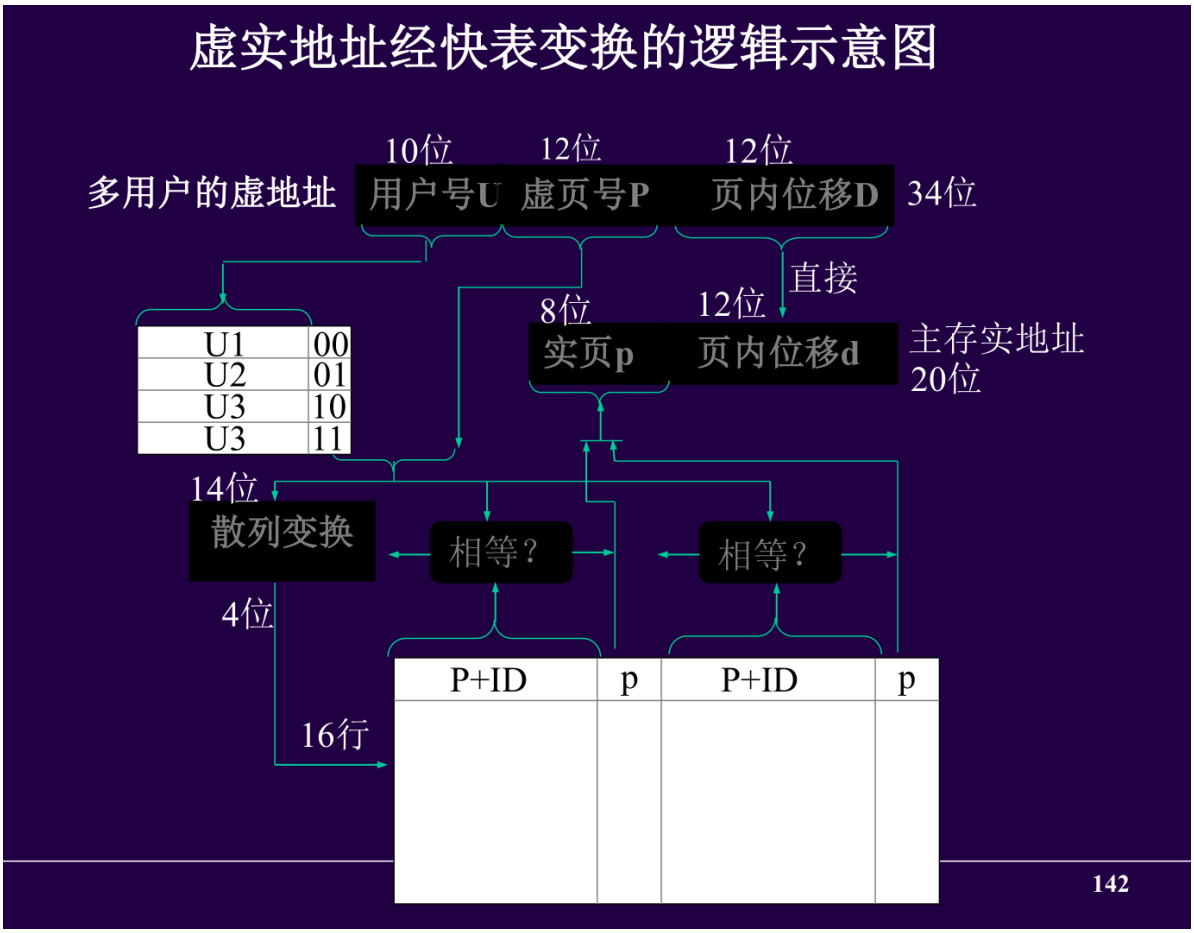
（5）每个相等比较器的位数为

（6）快表的总容量



CSDN @Code_流苏

确认一下, 是否要掌握散列变换



计算虚地址到实地址的映射时, 直接用虚地址模页面大小即可得到虚页号, 然后虚地址-虚页号*页面大小=页内偏移, 最后的实地址=实页号x页面大小+页内偏移. 此过程需要在十进制下完成, 如果有十六进制时可以转换为十进制.

例4.3 设某用户虚存共有8页，主存有4页，每页大小为1KB. 试根据页表计算出虚地址1023和6800的主存实地址。

页表

虚页号	实页号	装入位
0	3	1
1	1	1
2	2	0
3	3	0
4	2	1
5	1	0
6	0	1
7	0	0

提示：注意页表中虚、实页对应关系



解：实页号与地址对应关系

第0页	0—1023
第1页	1024—2047
第2页	2048—3071
第3页	3072—4095
第4页	4096—5119
第5页	5120—6143
第6页	6144—7167
第7页	7168--8191

虚页号＝虚地址%1024

虚地址1023，虚页号为0，页内位移为1023；根据虚页号查页表得知实页
号为3，且装入位为1。
主存实地址PA=3072+1023=4095

虚地址6800，虚页号为6，页内位移为656；根据虚页号查页表得知实页号为0，且装入位为1。
主存实地址PA=0+656=656

每页首地址=页号X每页大小

cache综合计算: 访问效率, 等效访问时间, 命中率, 块大小对命中率的提升

- 题4.1 在一个Cache存储系统中， Cache的访问周期为10ns，主存储器的访问周期为60ns，每个数据在Cache中平均重复使用4次。当块大小为1个字节时，存储系统的访问效率只有0.5，现在要通过增加块大小，使存储系统的访问效率达到0.94。
- （1）当存储系统的访问效率为0.5时，计算命中率和等效访问周期。
- （2）为了使存储系统的访问效率达到0.94，命中率和等效访问周期应该提高到多少？
- （3）为了使存储系统的访问效率从0.5提高到0.94，块的大小至少增加到几个字？

解：（1）当存储系统的访问效率为0.5时，由表达式

$$e=\frac{T_{A1}}{T_A}=\frac{T_{A1}}{HT_{A1}+(1-H)T_{A2}}=\frac{1}{H+(1-H)T_{A2}/T_{A1}}$$

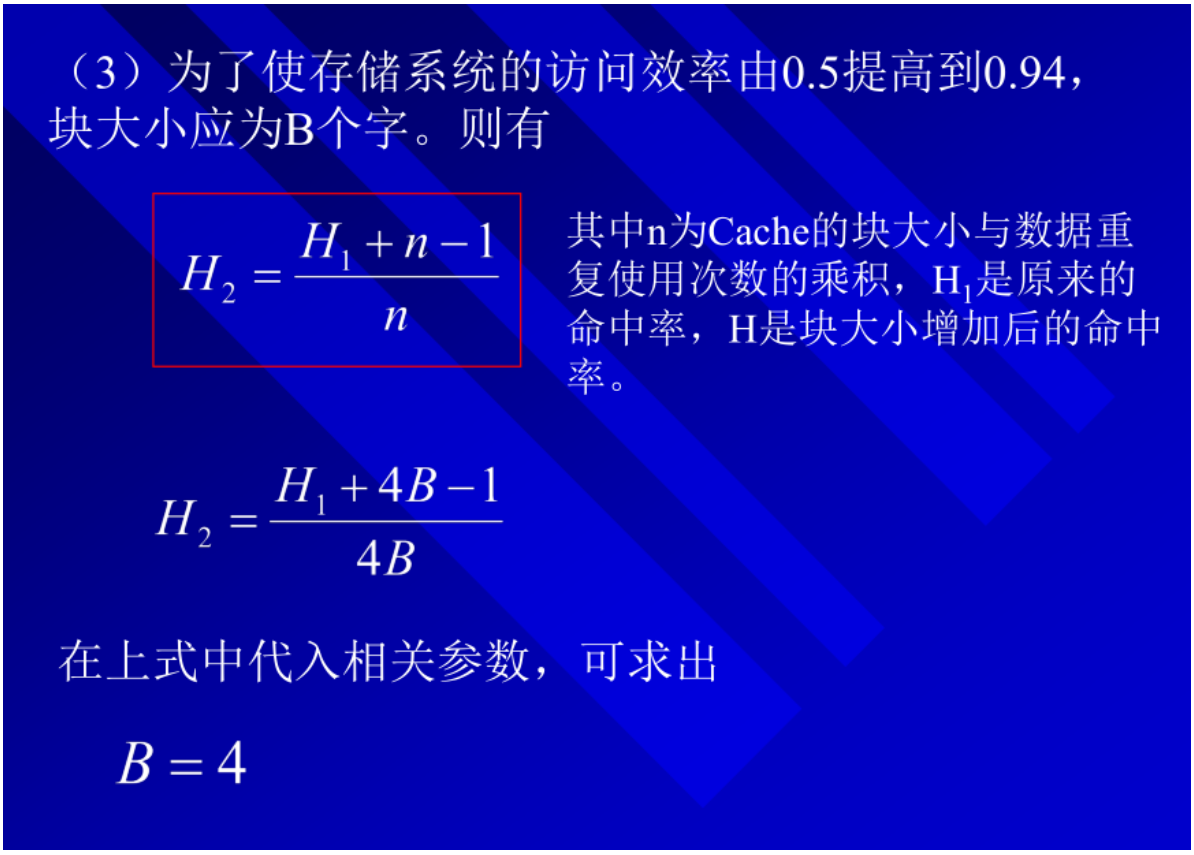
可求出命中率为

$$0.5=\frac{1}{H_1+(1-H_1)60/10} \qquad H_1=0.8$$

等效访问周期为

$$T_1=T_c\times H_c+(1-H_c)\times T_m$$
$$=10\times 0.8+60\times 0.2=20(ns)$$

或由 $e=\frac{T_{A1}}{T_A}=\frac{T_c}{T_1}=\frac{10}{T_1}=0.5$ 得 $T_1=20(ns)$

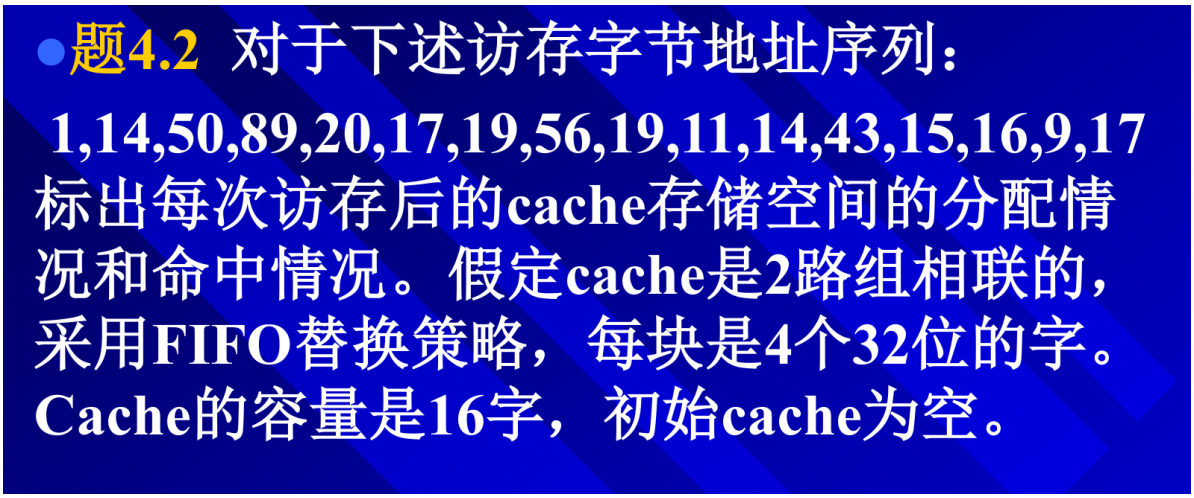


Q: 有没有便捷的方式计算主存地址到cache组号的映射

1. 先写出主存地址的格式, 然后看组号对应的位数的十进制是几
2. 算出主存地址对应的主存块号, 利用模运算得到对应的cache组号

假设按字编制, 则一块有4个地址, 用地址除以4即可得到对应块号:

例如 50/4=2




（2路组相联FIFO）
时间 t 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16
字地址流 1 14 50 89 20 17 19 56 19 11 14 43 15 16 9 17
组0 1 1 1 1* 20 20* 19 19 19 19 19 19 19 19 19* 17
50 50 50* 17 17 17 17 17 17 17 17* 16 16 16
组1 14 14 14 14 14* 56 56 56* 14 14* 15 15 15 15
89 89 89 89 89 89* 11 11* 43 43 43* 9 9
调进 调进 调进 调进 替换 替换 替换 替换 命中 替换 替换 替换 替换 替换 替换 替换
命中1次

为什么CPI这样计算?

某 RI SC 处理机的工作主频为 500MHz， 有一个 Cache 和一个主存储器， Cache的存取周期为 2ns， 主存储器的存取周期为 20ns， Cache 的命中率为 99%， 有 20%的 LOAD/STORE 指令， 并假设处理机速度的瓶颈完全在存储系统。

(1) 求理想情况(Cache 的命中率为 100%) 下的 CPI。

(2) 计算该 RI SC 处理机的实际 MI PS 速率。

(3) 如果处理机的工作主频提高到 1GHz， 计算实际 CPI 和 MI PS 速率。

- 解答：
- (1) CPI=(2ns+2ns×20%)×500MHz=1.2
- (2) MIPS=1/(2ns×99%+20ns×1%+2ns×20%×99%+20ns×20%×1%)
- =1/2.616ns=382.3
- (3) CPI=(2ns×99%+20ns×1%+2ns×20%×99%+20ns×20%×1%)×1GHz
- =2.616ns×1GHz=2.616
- MIPS=1/2.616ns=382.3

分离cache消除结构冲突

计算平均访问时间=访问cache的时间+不命中率*失效开销(访问cache+访问主存)

给定以下的假设，

（1）对指令Cache的访问占全部访问的75%，对数据Cache的访问占25%；

（2）Cache的命中时间为1个时钟周期，失效开销为50个时钟周期；

（3）在混合Cache中一次load或store操作访问Cache的命中时间都要增加1个时钟周期；

（4）32KB的指令Cache的失效率为0.39%，32KB的数据Cache的失效率为4.82%，64KB的混合Cache的失效率为1.35%；

（5）采用写直达策略，且有一个写缓冲器，并且忽略写缓冲器引起的等待。

要求：

（1）对于指令Cache和数据Cache容量均为32 KB的分离Cache，计算其失效率，且计算其平均访存时间；

（2）计算容量为64 KB的混合Cache的平均访存时间；

（3）分别从失效率和平均访存时间两个方面，比较分离Cache与混合Cache的性能

- 解：
- （1）对于分离Cache的总体失效率为：
- （75%×0.39%）＋（25%×4.82%）＝1.4975%
- 平均访存时间公式可以分为指令访问和数据访问两部分：
- 平均访存时间　＝　指令所占的百分比×（指令命中时间＋指令失效率×失效开销＋数据所占的百分比×（数据命中时间＋数据失效率×失效开销）
- 分离Cache的平均访存时间＝75%×（1+0.39%×50）+25%×（1+4.82%×50）
- ＝1.74875
- （2）混合Cache的平均访存时间＝75%×（1+1.35%×50）+25%×（1+1+1.35%×50）
- ＝1.925
- （3）从失效率方面来看，分离Cache为1.4975%，其大于混合Cache的失效率1.35%，混合Cache的性能优于分离Cache。
- 但从平均访存时间来看，分离Cache为1.74875个时钟周期，其小于混合Cache的1.925个时钟周期，分离Cache的性能优于混合Cache。
- 因此，尽管分离Cache的实际失效率比混合Cache的高，但分离Cache提供了两个端口，消除了结构冲突，其平均访存时间反而较低。

特殊的计算加速比的方法

作5.13 一台非流水处理机A的工作时钟频率为25MHz，它的平均CPI为4。处理器B是A的改进型，它有1条5段的线性指令流水线。由于锁定电路延迟及时钟扭斜效应，它的工作时钟频率仅为20MHz，问：

（1）若在A和B两个处理器上执行100条指令的程序，则处理器B对A的加速比是多少？

（2）在执行上述程时，计算A，B处理器各自的MIPS速率是多少？

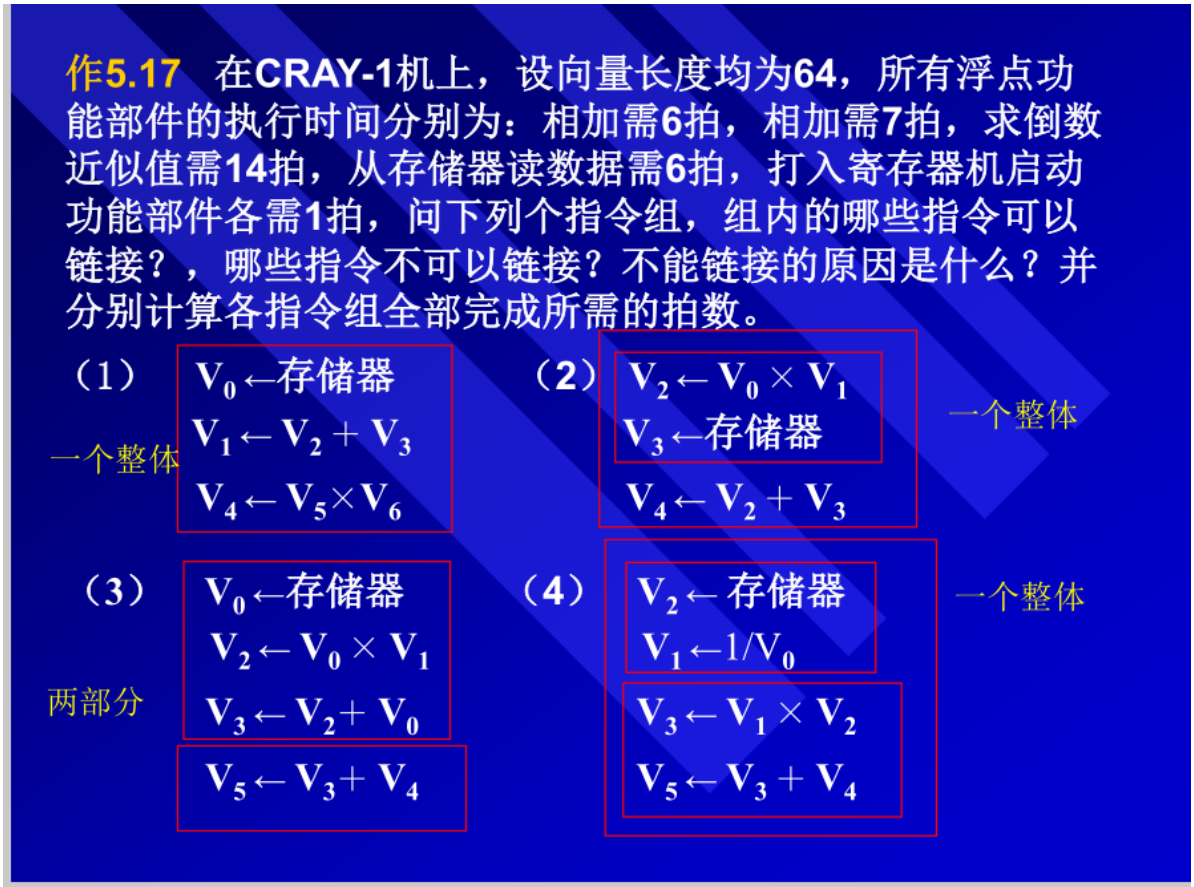
解：已知：$MIPS_B=20$

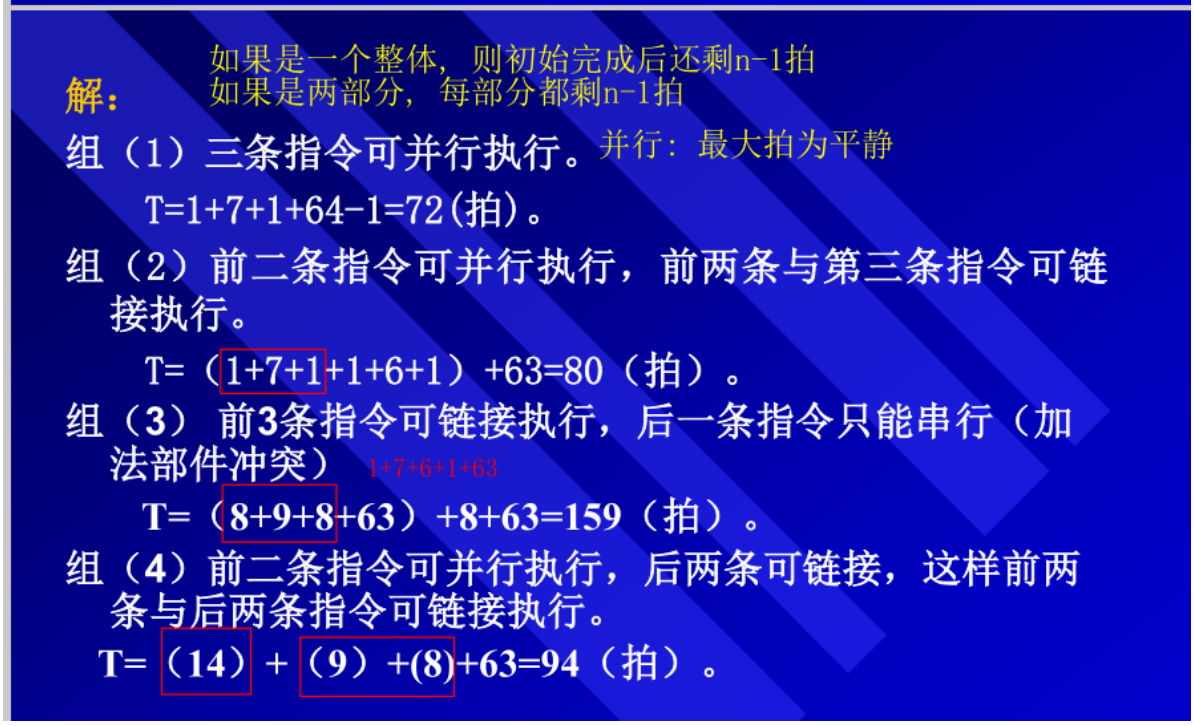
$$MIPS_A=\frac{R_C}{CPI\times10^6}=\frac{25M}{4M}=6.25$$

$$S_P=\frac{MIPS_B}{MIPS_A}=\frac{20}{6.25}=3.2$$

向量流水线

- 可并行
  - 数据独立
  - 操作类型不相同
- 可链接
  - 操作数直接传入到下一条指令





这里我们复习一下，CRAY-1向量流水处理部分的相关知识点：

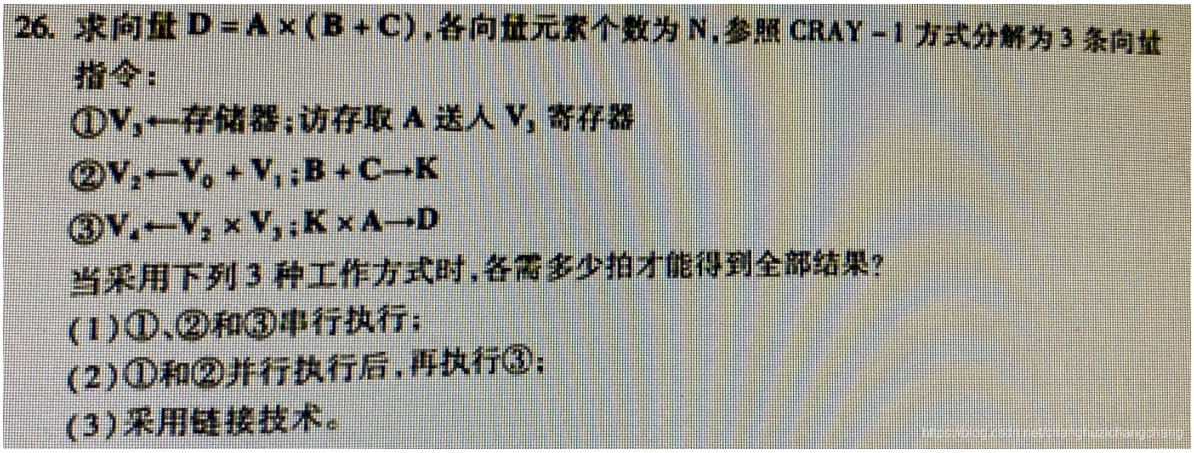
CRAY-1启动访存、把元素送往功能部件及结果存入Vi都需要1拍的传送延迟。

CRAY-1访存流水线的建立需要6拍，另外浮加6拍，浮乘7拍。

这些书上都有记载，都记在了犄角旮旯，系统结构的难度绝对是被这本教材拉上去的。

例题的巩固

ps:这个真题是哪年的，这图实在太糊了



（1）1、2、3串行执行需要多少拍?
第一条向量指令，V3<–存储器:
1拍启动访存+6拍访存+1拍访存结果存入V3，第一个结果分量就出来了，之后每过一拍就可以出一个结果，向量长度为N,全部出来要N-1拍。
结果分量1：(1+6+1）+N-1

第二条向量指令，V2<–V0+V1:
1拍送浮加+6拍浮加+1拍访存结果存入V2。
结果分量2：(1+6+1)+N-1

第三条向量指令，V4=V2 X V3
1拍送浮乘+7拍浮乘+1拍访存结果存入V4。
结果分量3：(1+7+1)+N-1
最终结果:
【(1+6+1)+N-1】+【(1+6+1)+N-1】+【(1+7+1)+N-1】=3N+22 拍

（2）1、2并行执行后，再执行3?
前两条并行执行，所以只用一个(1+6+1)+N-1的时间
最终结果:【(1+6+1)+N-1】+【(1+7+1)+N-1】=2N+15 拍

（3）采用链接技术?
链接技术，书上也有提及，如下图所示:

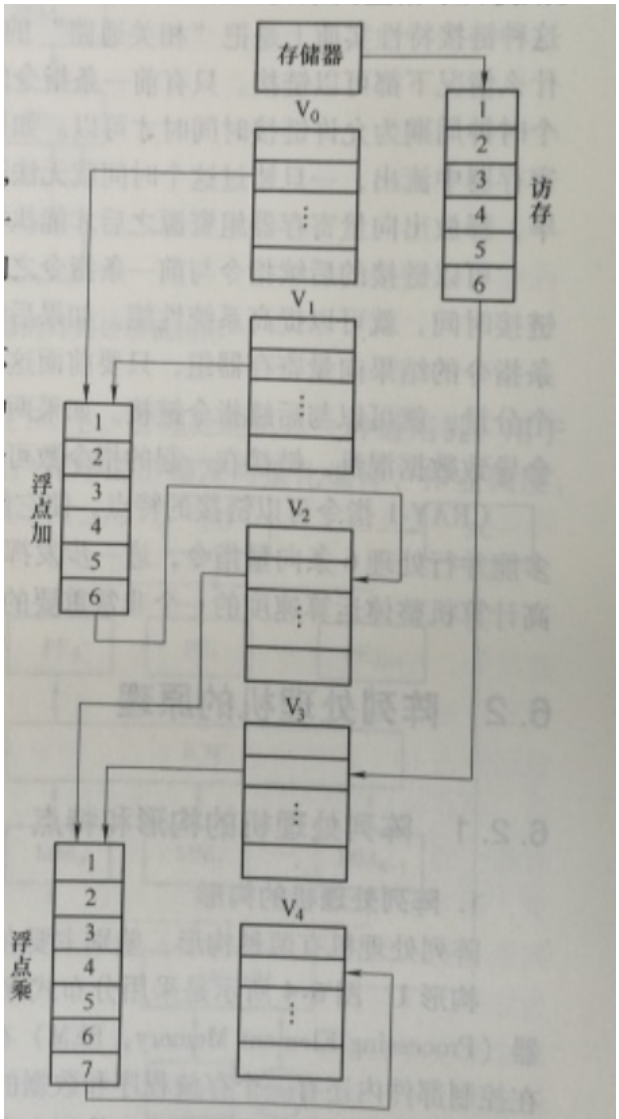
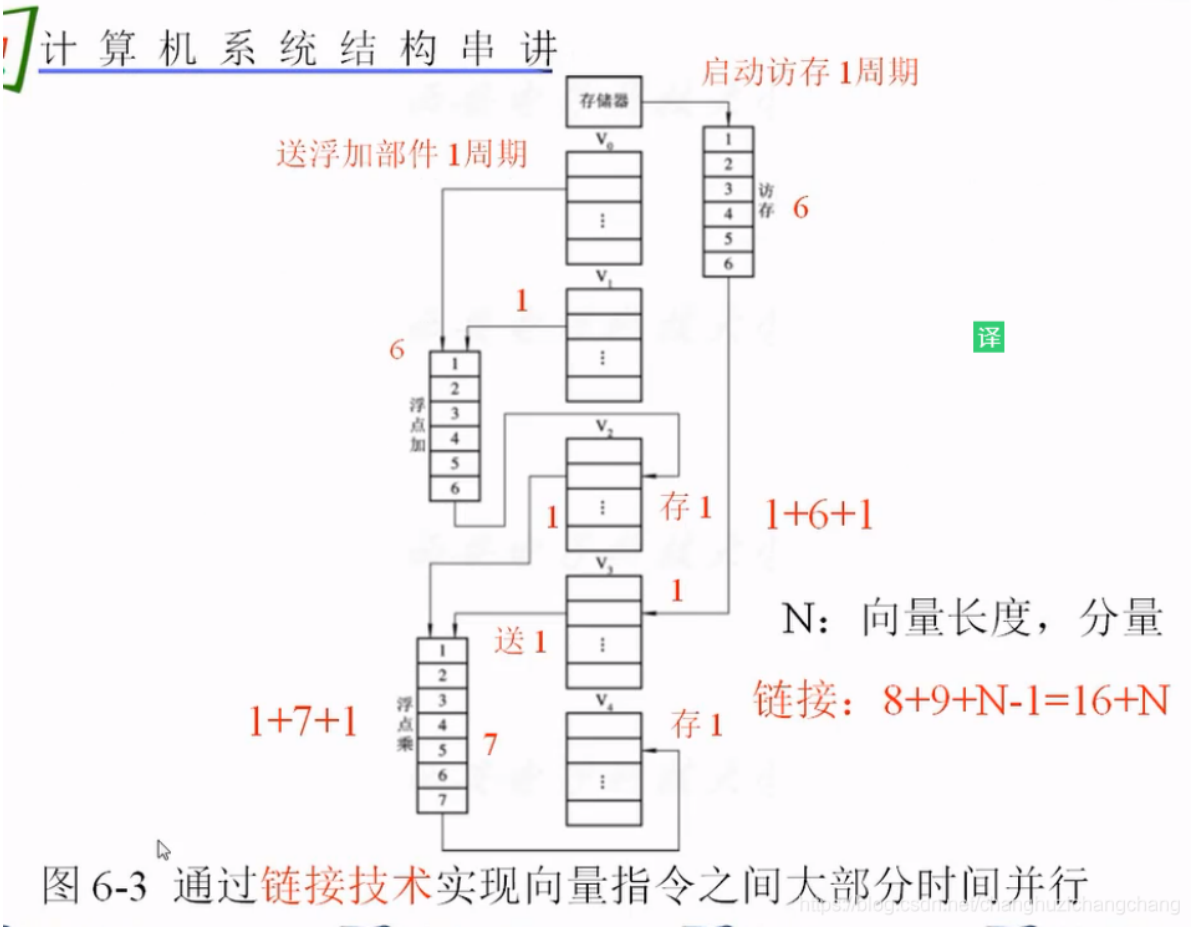


图 6-3　通过链接技术实现向量指令之间大部分时间并行

https://blog.csdn.net/changhuzichangchang



换个图更明白

访存和浮加并行，第一个V2分量和V3分量出来之后，不等下面的向量分量结果全部出来，而是直接把这两个值送去浮乘。之后每过一拍就可以出一个结果，向量长度为N,全部出来要N-1拍。
最终结果:(1+6+1)+(1+7+1)+N-1=N+16拍

————————————————————

版权声明：本文为CSDN博主「哑巴湖小水怪」的原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接及本声明。
原文链接：https://blog.csdn.net/changhuzichangchang/article/details/115523788

为什么3返回的时候要先到2再到4? 不能直接到4么?

32

132 // 1的优先级大于3, 所以加入1; 4的优先级小, 先不加入

32

2

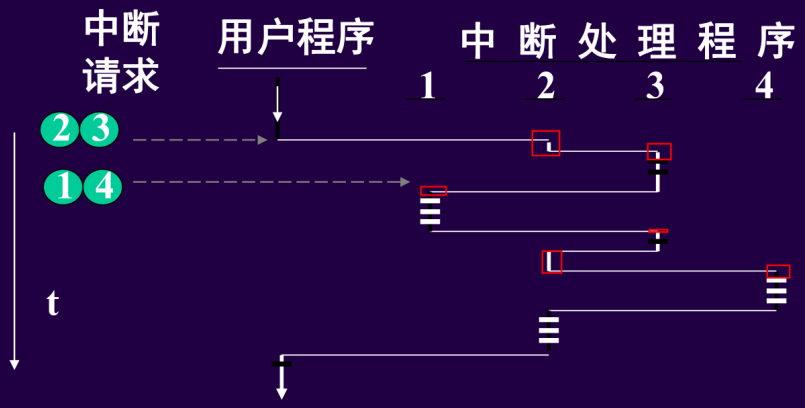
42 // 只有等3执行完该执行2的时候, 4才能加进去, 因为4的优先级只高于2

2

【解答】

(1) 当中断响应的优先次序为1、2、3、4，其中断处理次序是1、3、4、2。

(2) 运行过程示意图如下：

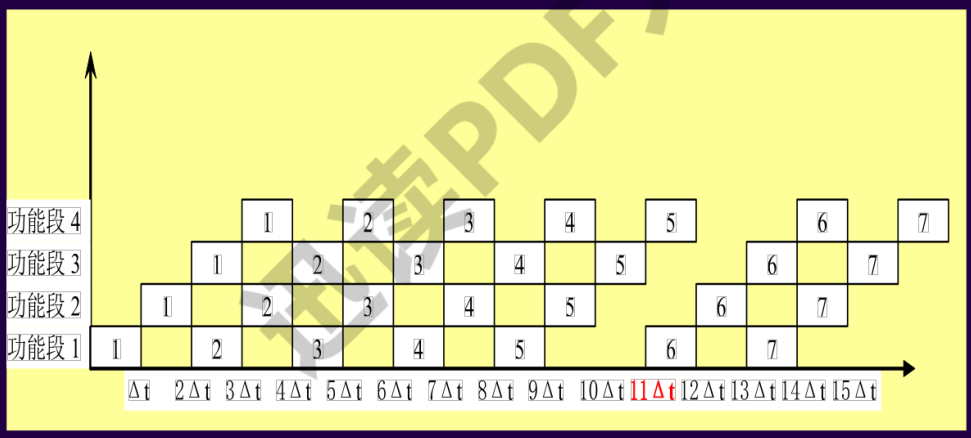


求极限

该水印由迅读PDF生成，
如果想去掉该水印，请访问并下载

作业5.7 p343 一条线性流水线有4个功能段组成，每个功能段的延迟时间都相等，都为Δt。开始5个Δt，每间隔一个Δt向流水线输入一个任务，然后停顿2个Δt，如此重复。求流水线的实际吞吐率、加速比和效率。

[解答]流水线的时空图如下：



迅读PDF大师

体系结构期末复习:

【计算机系统结构】2021期末复习考试重点大纲:https://blog.csdn.net/qq_46526828/article/details/118858843

计算机系统结构 | 期末复习总结学习笔记

计算机系统结构之重要知识点总结1: https://blog.csdn.net/qq_39382769/article/details/85241564

计算机系统结构复习总结: https://www.cnblogs.com/iforeverhz/p/16255934.html

计算机系统结构 学习笔记(期末复习总结) https://blog.csdn.net/weixin_30894389/article/details/95053332

【计算机系统结构】期末考试备考复习宝典 （必考考点--建议收藏）: https://blog.51cto.com/u_15270205/2991410

【计算机系统结构】期末考试备考复习宝典 （搞定五十个必考名词解释）: https://blog.51cto.com/u_15270205/3003948

计算机系统结构期末知识点总结(最后一课): https://blog.csdn.net/Ws_zqw/article/details/122023561

【2017/6/9】期末复习之计算机系统结构资料:https://zhuanlan.zhihu.com/p/27327055

重点:

1. 结合机器学习和深度学习的需求，体系结构应该怎么改变。
2. 填空（1×20） cache主存层次，更新算法；并行；简单的计算。
3. 问答题（第一讲——定性设计原则，**不同类型存储系统构造或者存储系统**）
4. 计算类型题目（三至十）：

向量处理机链接技术；标量处理机流水线，时空图（forward），结构相关，数据相关，控制相关，时间计算，性能计算；

输入输出系统：中断优先级（两种做法），几种类型的通道，通道流量的计算，估算可能会有多少台设备挂入，哪些能哪些不能；

存储系统：基本存储层次，采用层次化方法目的，存储系统设计的时候的计算，计算效能更好，置换算法（堆栈型的替换算法，模拟，页面地址流（这次不会这样））；

指令系统：指令的组成，指令的基本格式设计，指令的优化（操作码，操作数），指令系统的优化（RISC），设计相应的指令格式。

第一章

正向计算:

计算总CPI: 各种指令的比例(各种指令的数量)+各种指令的CPI(各种指令的时钟周期)

计算MIPS: CPI+CPU时钟频率

计算MFOLPS: MIPS+一个浮点数操作需要的时钟周期数

计算总执行时间: 指令总数量+平均CPI+时钟频率

计算CPU性能: 通过总执行时间衡量

反向计算:
时钟周期=CPI

时钟频率=MIPS=总时间

指令比例=CPI

问法:

性能提高了多少? 计算加速比

哪个CPU执行更快?

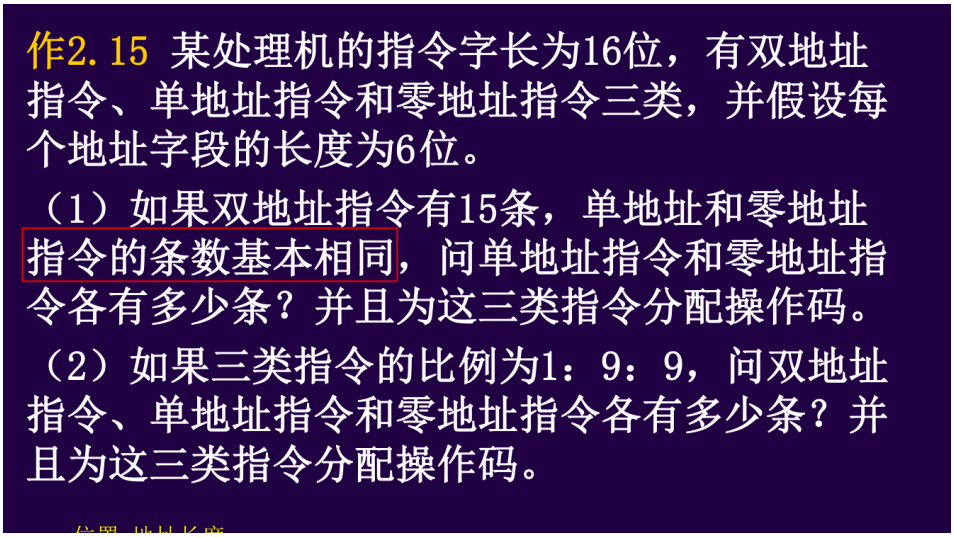
- 相同机器: 比较CPI
- 不同机器: 比较指令执行总时间

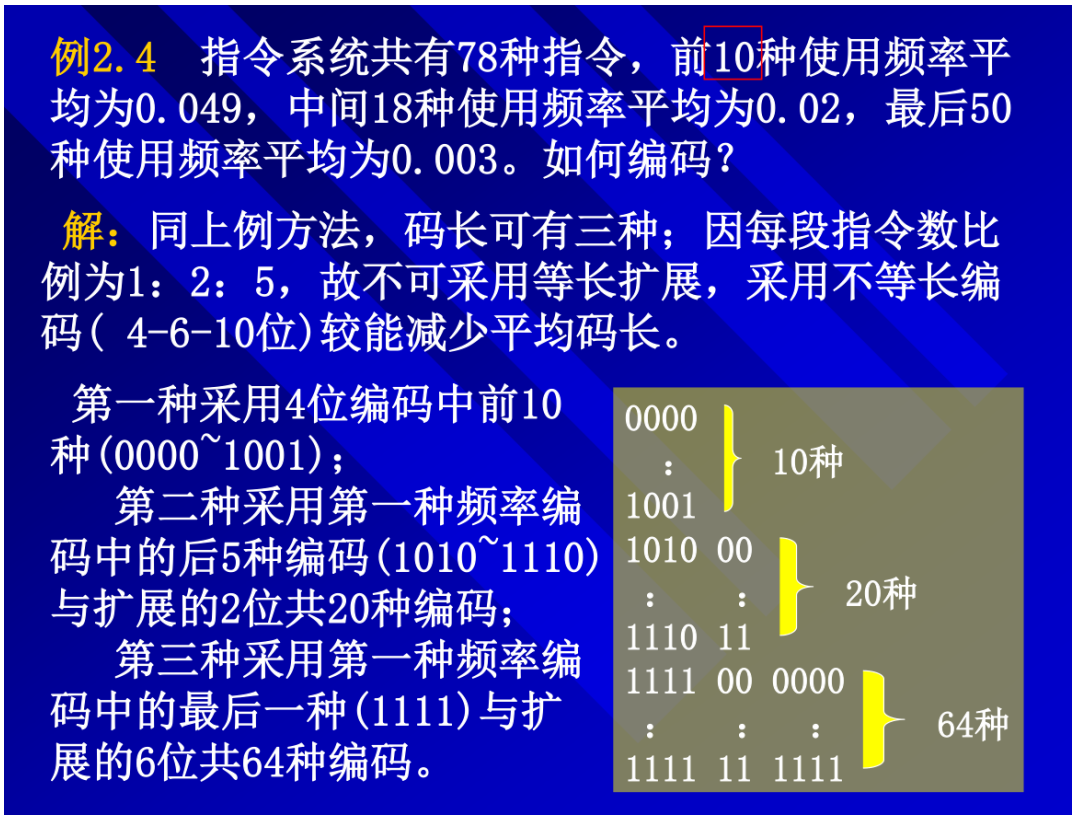
第二章

指令 = 操作码 + 操作数

指令频率决定了操作码的设计

- 哈夫曼编码
    - 编码长度计算
    - 最优码长计算=信息熵
    - 冗余度计算=1-哈夫曼长度/最优长度
    - **2-x编码; 3-x编码**
- **等长扩展编码**
    - 适用: 指令数量等比例增加, 如d:kd:2kd, 间隔为kd
    - 核心: 保证前缀不重复
        - 实现: 前缀扩展/标志位扩展
    - **4-8-12编码(等差数列编码)**
        - 特性:
            - 等差数列编码出来的不同指令可以通过前缀扩展编码使得数量相同(例如下面的15/15/15)



    - 数量也可以通过扩展标记位按照倍数增长
  - 15/15/15: 思想就是使用1111作为前缀扩展编码, 避免了4位指令和8位指令前缀的重复
    - 变种: 如果操作码长度为4的指令条数为小于15, 且长度为8和12的质量数量相等. 三种长度的指令满足比例m:n:n, 则设长度为4的指令剩余x个地址, (15-x)/15x=m/n. 其中15-x指的是长度为4的指令个数, 这点很好理解. 难点是为什么分母的长度为8的指令个数是15x. 其实这点很简单, **那是因为对于剩余的x个地址, 我们都分别将其当做是前缀进行扩展, 每次扩展都能产生15条以其为前缀的地址.**
  - 8/64/512: 思路就是使用扩展标识位标记每一段后续是否还有其他段, 每一段的长度是4, 第一位是标志位.
    - 变种: 根据频率最大指令的条数确定变种, 例如频率最大的指令只有4条, 则可以用3/6/9标记为扩展编码, 实现4/16/64编码. 0xx; 1xx 0xx; 1xx 1xx 0xx.
- 不等长扩展编码
  - 数量不成比例
  - 

  - 思路: 先根据频率最大的指令数量设计初始位数, 例如上面例题要包含10条指令就需要4位初始指令. 然后基于这4位指令中没有用完的质量继续扩展, 能够包含住下一种指令个数即可.
- 给出各种指令的数量和使用频率, 设计指令编码

- 首先根据数量估计出大致比例, 判断是否成比例
  - 成比例:
    - 套用15/15/15的前缀扩展编码
    - 套用8/16/512的标记位扩展编码
  - 不成比例:
    - 使用不等长扩展编码, 使得扩展长度后恰好可以包含指令数目即可.
- 使用频率最大的质量数量确定初始指令长度

操作数:

1. 寄存器(位数由寄存器个数决定)
2. 内存(位数由主存地址长度决定)
3. 立即数

【指令设计题目的套路(正向)】:

指令格式: **操作数**(哈夫曼/自己根据指令数量和频率设计) -- **寄存器** -- **内存**(根据寻址方式具体决定有几项)

指令概率 -> 哈夫曼编码(算出来的是操作数编码) -> 编码长度/冗余度...

-------除了哈夫曼编码, 还以用2-4编码等扩展编码进行指令设计, 具体用什么看不同指令的个数-------

指令条数/指令频率 -> 设计扩展编码的时候用, 先对频率进行类别划分. 如果两类指令数量相同，考虑使用类似4-8-12中15/15/15的前缀扩展编码。

寄存器个数 -> 寄存器位数

指令长度(指令字长) -> 设计指令时推导某一项长度用, 例如变址寄存器的位数=总位数-寄存器位数-偏移距离位数.

寻址方式 -> 内存访问在指令设计中的表示

- 间接寻址: 基址寄存器(固定的通用寄存器)+偏移地址(变化的, 需要长度推导)
- 变址寻址: 变址寄存器(变化的,需要长度推导) + 偏移量(固定的, 需要题目**变址范围**决定)

单/双/零地址+地址字段长度 -> 操作码长度=指令长度-地址字段长度*n

指令比例 -> 结合指令操作码长度计算某种指令固定扩展头时的数量, 并借助比例解方程算

(2)

假设双地址指令 x 条，则单地址、零地址分别为 9x 条：

$$[(2^4 - x) * 2^6 - 9x] * 2^6 = 9x$$

解之即得：x =14

假设双地址剩余x个地址没有用：
(16-x)/63x = 1/9
解：x=2

∴ 双地址指令 14 条，操作码为：0000~1101；留出两个编码用于扩展。

单地址指令$(2^6-1)\times 2 = 126$ 条，操作码为：

1110 000000~1110 111110，1111 000000~1111 111110

零地址指令 126 条，操作码为：

1110 111111 000000~1110 111111 111110，1111 111111 000000~1111 111111 111110

第三章

存储系统: 多级.不同速度

虚拟内存

- 将虚拟地址映射到物理地址
  - 解析虚拟地址 (页大小决定业内偏移大小->页内偏移位数; 虚存大小->虚存中总页数->页号位数)
    - 分页
      - 页号+页内偏移
      - 多级页表..
    - 分段
      - 段号+段内偏移
    - 段页式
      - 段号+页号+页内偏移
    - 多用户
      - 用户编号+段号+页号+页内偏移
  - 匹配表项
    - 页表
    - 段表
    - 段表+页表
    - 用户表
    - 块表
  - 不命中: 缺页 -> 从磁盘中读取到内存中
    - 页面替换算法
  - 构成物理地址
- 因为内存和磁盘速度相差太大, 所以应该用**全相连**, 增加命中率

cache

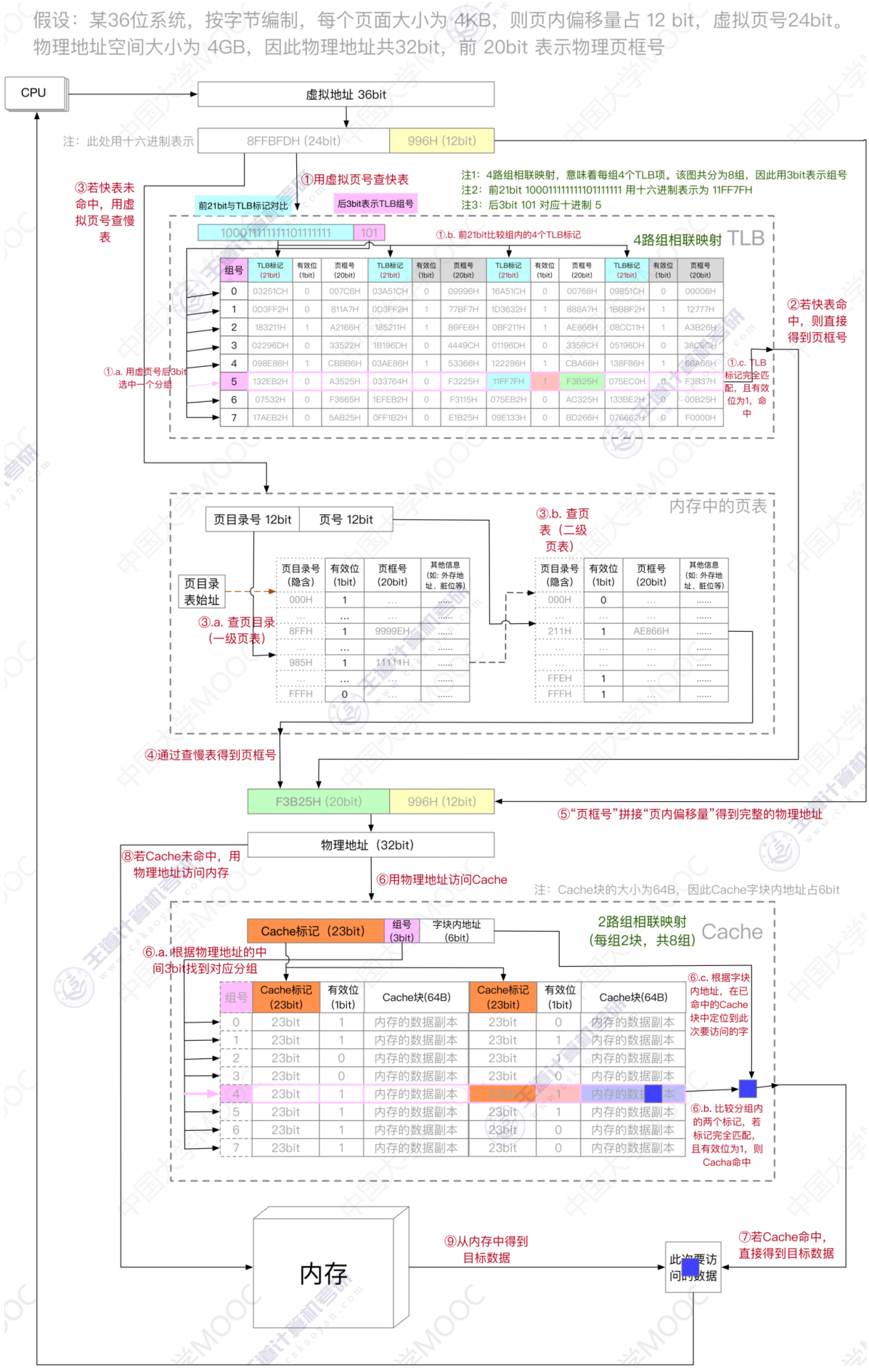
- 加速比, 等效时间, 访问效率, 命中率提升公式
- cache的工作过程
    - 物理地址到cache地址的映射 (物理地址的设计方式和cache地址的设计方式)
        - 物理地址的解析: 物理地址=块号+块内偏移
        - 根据相联方式的不同, 块号也要几种不同的分解方式:
            - 全相连: 块号就是每一块的编号, tag=块号
            - 直接相连: 块号=区号+区内块号, tag=区号
            - 组相连: 块号=区号+组号+组内块号, tag=区号+组内块号
        - 解析完成后, 与块表匹配找到cache地址:
            - 全相连: 直接用块号匹配
            - 直接相连: 用区内块号取模匹配找到表项, 然后取出**区号**与地址中的区号对比
            - 组相连: 用组号匹配找到表项, 然后取出G(=组内块数)个条目, 用其**区号+组内块号**和地址中的区号+组内块号匹配.
    - 从cache地址中读出数据
    - 如果未命中, 则直接用物理地址从主存读取数据, 并将读取的内存块放入cache
        - 缺页替换算法:LRU-堆栈型-n增大命中率不减少
            - 堆栈模拟算法: 可以得到最大命中率时的最小n
        - cache有空余位置可以直接放入, 但是注意, 如果是组相连, 则块放入的位置是有讲究的, 不能顺序放入.

虚拟存储+cache

- 都是两级不同速度的存储器构成的存储结构
    - 速度是最快的
    - 容量是最大的
- 都有地址映射的过程
    - 根据不同情况对地址进行初步解码
        - cache: 相联方式
        - 虚存: 分页分段
    - 匹配表项
        - cache: 块表/目录表
        - 虚存: 页表/块表/段表
    - 替换策略
        - LRU/LFU/FIFO/OPT
- 地址位数的计算往往和数量有关
    - cache
        - 块的数量(区内块/组内块)
        - 组的数量
        - 区的数量
    - 虚存
        - 页的数量
    - 512=9; 32=5; 64=6
- 编址方式不同:

- cache: 内存和cache独立编址
- 虚存: 统一编址逻辑地址

https://blog.csdn.net/weixin_43362002/article/details/120477957



假设LFU方法的时期T为10分钟，访问如下页面所花的时间正好为10分钟，内存块大小为3。若所需页面顺序依次如下：

2 1 2 1 2 3 4

---------->

当需要使用页面4时，内存块中存储着1、2、3，内存块中没有页面4，就会发生缺页中断，而且此时内存块已满，需要进行页面置换。
若按LRU算法，应替换掉页面1。因为页面1是最长时间没有被使用的了，页面2和3都在它后面被使用过。
若按LFU算法，应换页面3。因为在这段时间内，页面1被访问了2次，页面2被访问了3次，而页面3只被访问了1次，一段时间内被访问的次数最少。

原文链接：https://blog.csdn.net/weixin_43240734/article/details/123159387

第四章

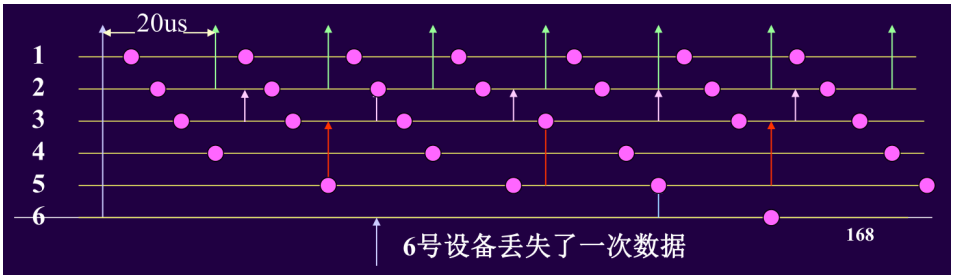
- 通道
  - 类型
    - 字节多路通道
      - 分时, 每个设备只传输一个字节
      - 中低速设备
    - 选择通道
      - 每个设备独占通道, 一次性把n个字节全部传送
      - 高速设备
    - 数组多路通道
      - 每个设备独占通道, 一次性传输k个字节. 传送完毕后分时切换, 一共切换n/k次可以把数据传输完成
      - 优化选择通道的处理等待问题
  - 计算
    - T: 完成全部数据传输任务所需的时间(所有设备所有数据)
    - n: 每个设备需要传输的字节数
    - Ts: 选择通道的时间
    - TD: 传输一个字节需要的的时间
    - **fMAX: 最大流量=单位时间传输的最大数据量** 用Ts TD n计算
    - **f: 实际传输的流量** 用时间间隔算
    - **△t: 时间间隔,** 每隔 △t 发出一个字节数据传送请求. [S/B]
      - 字节多路通道
        - T = P(nTs + **nTD**)
        - fMAX = n/T
        - f = 各设备传输速率之和
        - 最大接入的设备数量: 保证△t时间间隔内这些设备都能传输数据, 即T<=△t, P为设备数
      - 选择通道

        - T = P(Ts + **n*TD**)
        - fMAX = n/T
        - f = max(各设备传输速率) [B/S]
        - fi = 1/△t 知道时间间隔(S/B), 可以求出实际流量f(B/S)
        - 例4.3 设通道在数据传送期中，选择设备需4.9μS，传送一个字节数据需0.1μS。

          （1）其低速设备每隔250μS发出一个字节数据传送请求，问最多可接多少台这种设备？

          （2）若有A～E共5种高速设备，要求字节传送的间隔时间如下表所示，其时间单位为μS。若一次通信传送的字节数不少于1024个字节，问哪些设备可挂在此通道上？哪些则不能？

设备	A	B	C	D	E
时间间隔（μS）	0.13	0.1	0.11	0.2	0.3

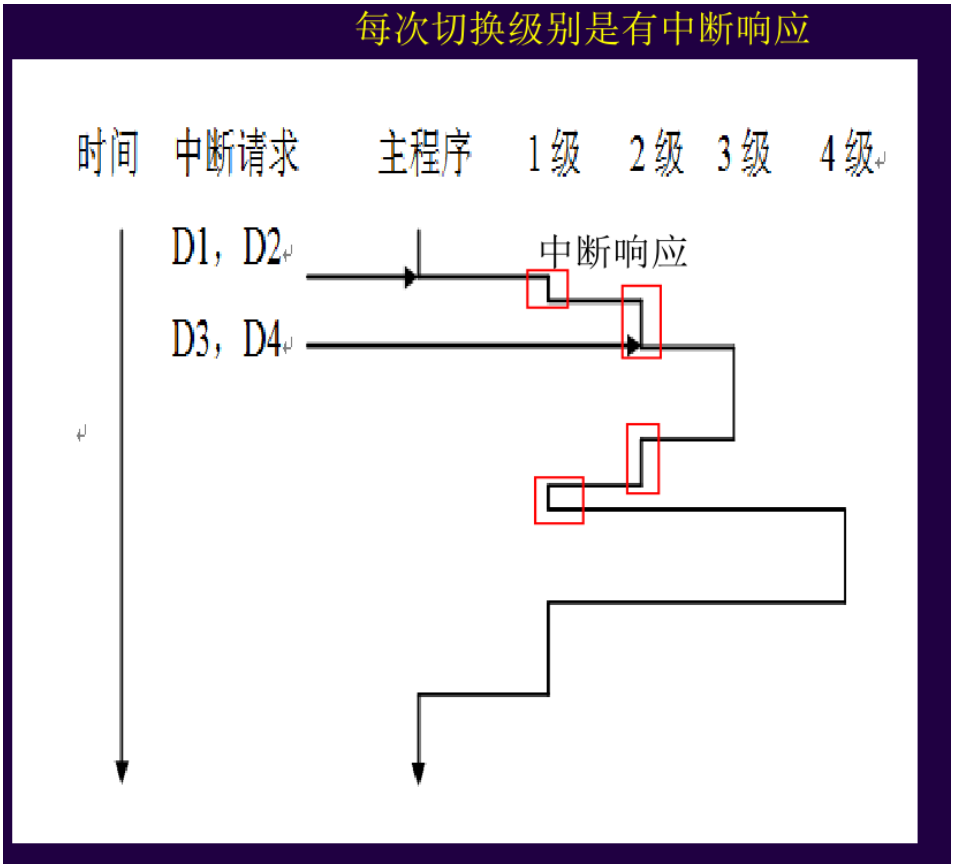
          p243
        - 数组多路通道
          - T = P(n/k*Ts + **n*TD**)
          - fMAX = n/T
          - f = max(各设备传输速率)
        - **保证通道正常工作的条件: f ≤ fMAX**
          - fMax通过题目给定的Ts TD n(一次性传输的字节数)计算
          - f 通过题目给定的时间间隔计算
      - 最大流量计算
    - 画图
      - 通道时间关系图
        - 

      - 慢设备传送数据失败改进方式
- 中断
  - 中断的类型
  - 中断优先级
    - 中断屏蔽码

- 不能屏蔽所有　可以屏蔽所有

	D1	D2	D3	D4
D1	1	0	0	0
D2	1	1	0	1
D3	1	1	1	1
D4	1	0	0	1

  - 运行过程示意图
    - 中断响应时间是按照中断到来的顺序的, 但是中断处理时间是按照中断屏蔽码指定的顺序来的.
    - 中断响应时间(非常小)
    - 中断处理时间
    - 每次切换级别是有中断响应



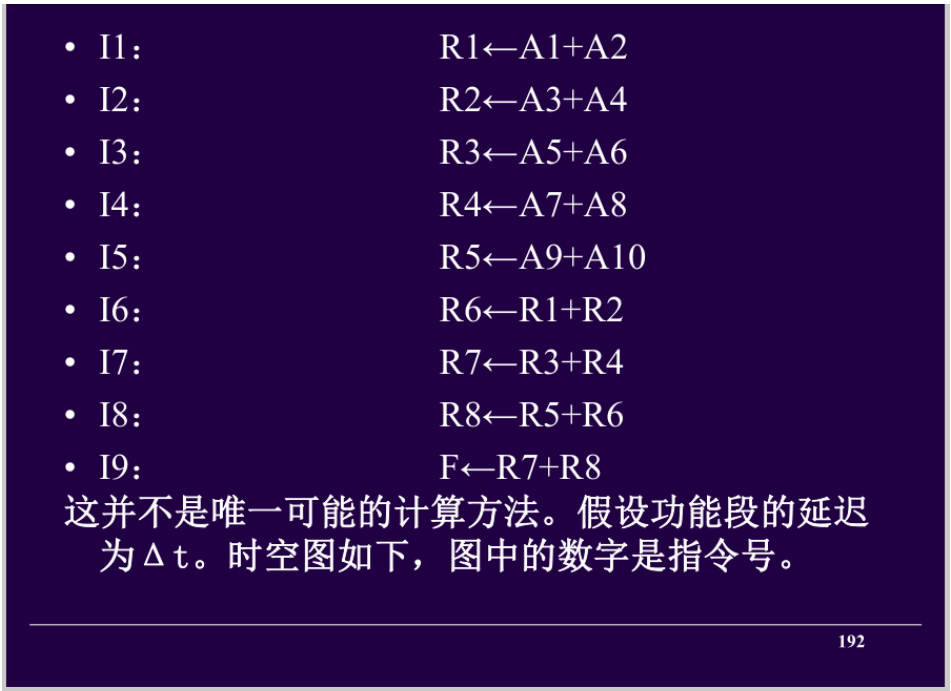
第五章

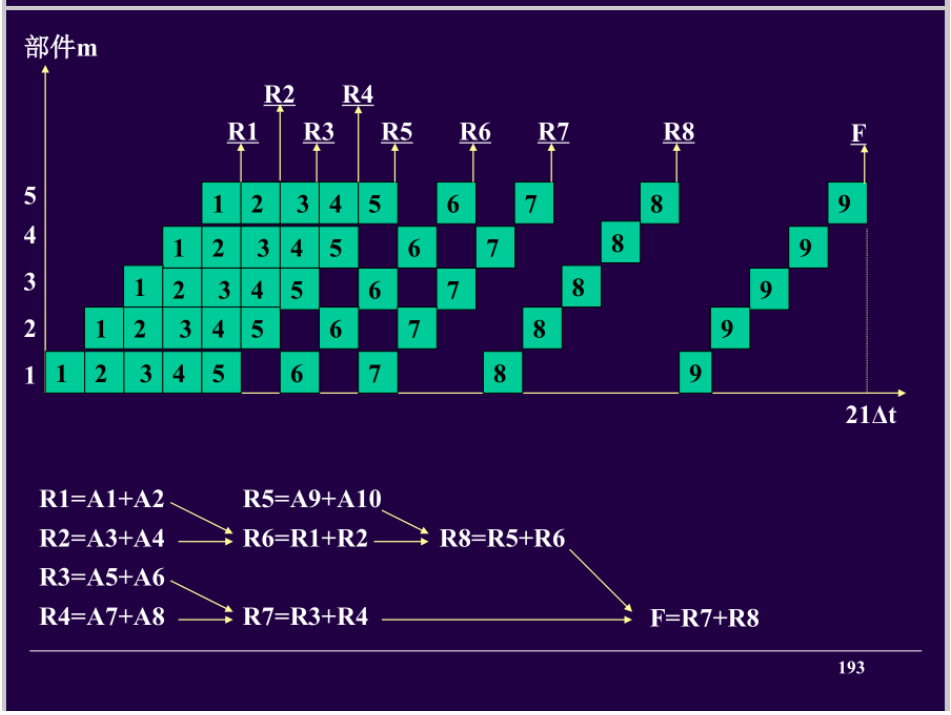
- 流水线的性能计算
  - 时空图

1. 静态流水线还是动态流水线
   - 静态: 某个类型的指令的流水线都出结果后, 再开启下一个类型指令的流水线
     - **同种类型的, 能重叠**
   - 动态: 只要某个流水线需要用到的输入数据已经获得, 则可以直接开启此流水线
     - **不同类型, 也可以重叠**

2. 归类操作类型, 注意数据相关

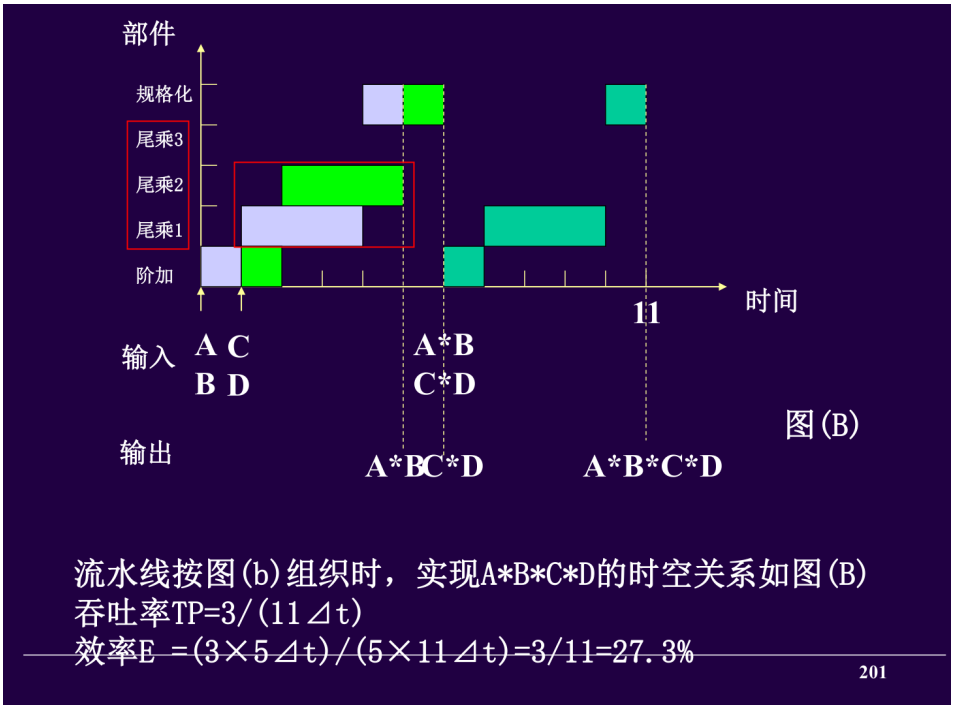
- 不同操作类型用不同的流水线处理, 其之间可能有数据相关 此时需要考虑流水线是静态还是动态
- 加法 / 乘法



- 

3. 对于长度不等的段

- 如果操作类型相同(如都是加法), 此时可以并行处理最长段p77，否则不行

- 

    - 吞吐量: 任务数/流水线总时间
    - 效率: 有效面积/总面积
    - 加速比: 串行时间/流水线并行时间
- 性能分析
    - 流水线擅长处理 **独立的/连续的** 任务
    - 段长的影响
        - 段长不是越长越好 p81
    - 为什么某个流水线性能差
        - 受限于最长的一段
        - 通过和完成时间占比大
        - 寄存器延时和时钟扭曲
    - 解决性能的方法
        - 瓶颈段处理方法
            - 细分
            - 重复设置
        - 将**同种类型**的操作放在一起

题目整理

例1：在一个页式二级虚拟存贮器中，采用FIFO算法进行页面替换，发现命中率H太低，因此有下列建议：
(1) 增大辅存容量
(2) 增大主存容量(页数)
(3) 增大主、辅存的页面大小
(4) FIFO改为LRU
(5) FIFO改为LRU，并增大主存容量(页数)
(6) FIFO改为LRU，且增大页面大小
试分析上述各建议对命中率的影响情况。

[解答]
(1) 增大辅存容量，对主存命中率H不会有什么影响。因为辅存容量增大，并不是程序空间的增大，程序空间与实主存空间的容量差并未改变。所以，增大物理辅存容量，不会对主存的命中率H有什么影响。

(2) 如果主存容量(页数)增加较多时，将使主存命中率有明显提高的趋势。但如果主存容量增加较少，命中率片可能会略有增大，也可能不变，甚至还可能会有少许下降。这是因为其前提是命中率H太低。如果主存容量显着增加，要访问的程序页面在主存中的机会会大大增加，命中率会显著上升。但如果主存容量(页数)增加较少，加上使用的FIFO替换算法不是堆栈型的替换算法，所以对命中率的提高可能不明显，甚至还可能有所下降。

(3) 因为前提是主存的命中率H很低，在增大主、辅存的页面大小时，如果增加量较 小，主存命中率可能没有太大的波动。因为FIFO是非堆栈型的替换算法，主存命中事可能会有所增加，也可能降低或不变。而当页面大小增加量较大时，可能会出现两种相反的情况。当原页面大小较小时，在显著增大了页面大小之后，一般会使主存命中率有较大提高。但当原页面大小已较大时，再显著增大页面大小后，由于在主存中的页面数过少，将会使主存命中宰继续有所下降。

(4) 页面替换算法由FIFO改为LRU后，一般会使主存的命中率提高。因为LRU替换算法比FIFO替换算法能更好地体现出程序工作的局部性特点。然而，主存命中率还与页地址流、分配给主存的实页数多少等有关，所以，主存命中率也可能仍然较低，没有明显改进。

(5) 页面替换算法由FIFO改为LRU，同时增大主存的容量(页数)，一般会使主存命中率有较大的提高。因为LRU替换算法比FIFO替换算法更能体现出程序的局部性，又由于原先主存的命中宰太低，现增大主存容量(页数)，一般会使主存命中率上升。如果主存容量增加量大些，主存命中率H将会显著上升。

(6) FIFO改为LRU，且增大页面大小时，如果原先页面大小很小，则会使命中率显著上升；如果原先页面大小已经很大了，因为主存页数进一步减少而使命中率还会继续有所下降。

例2：采用组相联映象、LRU替换算法的Cache存贮器，发现等效访问速度不高，为此提议：
(1) 增大主存容量
(2) 增大Cache中的块数(块的大小不变)
(3) 增大组相联组的大小(块的大小不变)
(4) 增大块的大小（组的大小和Cache总容量不变）
(5)提高Cache本身器件的访问速度
试问分别采用上述措施后，对等效访问速度可能会有什么样的显著变化？其变化趋势如何？如果采取措施后并未能使等效访问速度有明显提高的话，又是什么原因？

[分析] Cache存储器的等效访问时间 ta = Hc*tc* + *(1-Hc)*tm

等效访问速度不高，就是ta太长。要想缩短ta，一是要使Hc命中率尽可能提高，这样(1-Hc)tm的分量就会越小，使ta缩短，越来越接近于tc。但如果ta已非常接近于tc时，表明Hc已趋于1，还想要提高等效访问速度，则只有减小tc，即更换成更高速的Cache物理芯片，才能缩短ta。另外，还应考虑Cache存贮器内部，在查映象表进行Cache地址变换的过程时，是否是与访物理Cache流水地进行，因为它也会影响到ta。当Hc命中率已很高时，内部的查映象表与访Cache由不流水改成流水，会对tc有明显的改进，可缩短近一半的时间。所以，分析时要根据不同情况做出不同的结论。

如果Cache存贮器的等效访问速度不高是由于Hc太低引起的，在采用LRU替换算法 的基础上，就要设法调整块的大小、组相联映象中组的大小，使之适当增大，这将会使Hc 有所提高．在此基础上再考虑增大Cache的容量．Cache存贮器中，只要Cache的容量比较 大时，由于块的大小受调块时间限制不可能太大，增大块的大小一般总能使Cache命中率 得到提高．

[解答]
(1)增大主存容量，对Hc基本不影响．虽然增大主存容量可能会使tm稍微有所加大，如果Hc已很高时，这种tm的增大，对ta的增大不会有明显的影响。

(2)增大Cache中的块数，而块的大小不变，这意味着增大Cache的容量．由于LRU替换算法是堆栈型的替换算法，所以，将使Hc上升，从而使ta缩短．ta的缩短是否明显还要看当前的Hc处在什么水平上．如果原有Cache的块数较少，Hc较低，则ta会因Hc迅速提高而显著缩短．但如果原Cache的块数已较多，Hc已很高了，则增大Cache中的块数，不会使Hc再有明显提高，此时其ta的缩短也就不明显了。

(3)增大组相联组的大小，块的大小不变，从而使组内的块数有了增加，它会使块冲突 突概率下降，这也会使Cache块替换次数减少．而当Cache各组组内的位置已全部装满了 主存的块之后，块替换次数的减少也就意味着Hc的提高。所以，增大组的大小能使Hc提高，从而可提高等效访问速度。不过，Cache存贮器的等效访问速度改进是否明显还要看目前的Hc处于什么水平。如果原先组内的块数太少，增大组的大小，会明显缩短ta；如果 原先组内块数已较多，则ta的缩短就不明显了。

(4)组的大小和Cache总容量不变，增大Cache块的大小，其对ta影响的分析大致与(3) 相同，会使ta缩短，但要视目前的Hc水平而定。如果Hc已经很高了，则增大Cache块 的大小对ta的改进也就不明显了。

(5)提高Cache本身器件的访问速度，即减小ta，只有当Hc命中率已很高时，才会显著缩短ta。如果Hc命中率较低时，对减小ta的作用就不明显了。

————————————————————

版权声明：本文为CSDN博主「zhouie」的原创文章，遵循CC 4.0 BY-SA版权协议，转载请附上原文出处链接及本声明。
原文链接：https://blog.csdn.net/jave_f/article/details/79006394/